

📌 SMART NOTES

Chart.js + Productivity Dashboard

Beginner → Advanced | Hinglish Style 📌

By Anand • Full Notes • Chart.js Masterclass

Contents

- 📌 Ch 1-2 — Basics & Setup
- 📌 Ch 3-4 — Data Deep Dive & Chart Types
- 📌 Ch 5-6 — Options, Styling & Real Project Thinking
- 📌 Ch 7-8 — Dynamic Update & localStorage
- 📌 Ch 9 — Dashboard Architecture
- Ch 10-11 — Advanced Concepts & Canvas vs DOM
- 📌 Ch 12 — Project-Level Thinking + Summary

❓ CHAPTER 1: Basics (Zero Level)

❓ Chart kya hota hai?

Chart = Data ko visually dikhane ka tarika

Numbers ko graph mein convert karte hain — samajhna easy ho jaata hai ❓

- 10, 20, 30 ✗ (boring numbers — kuch samajh nahi aata)
- Bar Graph ✔ (ek second mein trend dikh jaata hai!)

❓ Chart.js kya hai?

JavaScript library hai jo graphs banata hai — aur canvas pe draw karta hai.

- Free & open source library
- 8 types of charts — bar, line, pie, doughnut, radar...
- Responsive + animated by default ✨

❓ 1-LINE DEF

"Data ko visual bana deta hai" — bas itna yaad rakh!

❓ Canvas kya hota hai?

Canvas = HTML ka drawing area / whiteboard ❓

```
<canvas id="myChart"></canvas>
<!-- Ye sirf ek khaali drawing board hai -->
<!-- Chart.js isi pe apna magic karta hai -->
```

❓ CHAPTER 2: Basic Setup

❓ Step 1: CDN add karo

```
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
<!-- Ye line <head> ya </body> se pehle daalo -->
```

❓ Step 2: Pehla Chart banao

```
let ctx = document.getElementById('myChart');

new Chart(ctx, {
  type: 'bar',           // chart ka type
  data: {
    labels: ['Mon', 'Tue', 'Wed'],
    datasets: [{
      label: 'Tasks Done',
      data: [5, 10, 7]
    }]
  }
});
```

❓ CHAPTER 3: Data Deep Understanding

❓ Data Structure

```
data: {  
  labels: ['Jan', 'Feb', 'Mar'], // x-axis ke naam  
  datasets: [{  
    label: 'Revenue', // legend ka naam  
    data: [1000, 1500, 1200], // actual values  
    backgroundColor: '#4CAF50'  
  }]  
}
```

❓ Kya matlab hai har cheez ka?

Property	Kya karta hai
labels	X-axis pe dikhne wale names
datasets	Actual data ka container (array)
label (dataset)	Legend mein dikhne wala naam
data (dataset)	Number values — inhi ko plot karta hai
backgroundColor	Bar / slice ka colour

❓ Multiple Datasets (Multiple Lines/Bars)

```
datasets: [  
  { label: '2024', data: [10, 20, 15], borderColor: 'blue' },  
  { label: '2025', data: [15, 25, 30], borderColor: 'red' }  
]  
// Result: 2 alag lines/bars ek hi chart mein! ☐
```

❓ YAAD RAKH

Ek dataset = ek line/bar group. Multiple = comparison!

❓ CHAPTER 4: Chart Types

Type	Use Case	Example
bar	Comparison / ranking	Monday vs Tuesday tasks
line	Trend over time	Weekly performance growth
pie	Distribution (parts of whole)	Time spent on subjects
doughnut	Pie ka better version ☐	Todo complete vs pending
radar	Multiple skill comparison	Skill assessment chart
polarArea	Proportional comparison	Study hours per subject

❓ PRO TIP

Doughnut > Pie — zyada readable hota hai center space ki wajah se!

❓ CHAPTER 5: Options & Styling

```
options: {
  responsive: true,           // mobile-friendly automatically
  plugins: {
    legend: {
      display: true,          // labels dikhao/chhupao
      position: 'top'         // 'top' | 'bottom' | 'left' | 'right'
    },
    title: {
      display: true,
      text: 'My Dashboard'    // chart ka title
    }
  },
  scales: {
    y: { beginAtZero: true }, // y-axis 0 se shuru
    x: { grid: { display: false } }
  },
  animation: { duration: 1000 } // 1 second animation
}
```

❓ Important Options — Quick Reference

Option	Kya karta hai
<code>responsive: true</code>	Chart khud resize hota hai screen ke hisaab se
<code>beginAtZero: true</code>	Y-axis 0 se start — galat impression nahi deta
<code>legend.display</code>	True = labels dikhta hai, false = chhupta hai
<code>animation.duration</code>	Chart draw hone ka time (milliseconds mein)
<code>backgroundColor</code>	Fill colour (bar/pie/doughnut mein)
<code>borderColor</code>	Line ka colour (line chart mein)
<code>borderWidth</code>	Line/bar ka border thickness

❓ CHAPTER 6: Real Project Thinking

✖ Beginner Approach

- Sirf UI banao
- Static data
- Ek feature = done

✔ Advanced Thinking

- Data track karo
- Usko analyze karo
- Chart mein dikhao = Insight!

❓ UPGRADE

Todo + Chart = "Analytics System" — resume pe zyada impact! ❓

❓ CHAPTER 7: Dynamic Update (IMPORTANT ❓)

Chart ko live update karna hai? Bas 2 steps:

```
// Step 1: Data push karo
chart.data.datasets[0].data.push(newValue);

// Step 2: Chart ko re-render karo
chart.update();

// Example: Button click pe update
document.getElementById('addBtn').addEventListener('click', () => {
  let val = parseInt(Math.random() * 100);
  chart.data.labels.push('New');
  chart.data.datasets[0].data.push(val);
  chart.update();
});
```

❓ Real-World Use Cases

- *Live dashboard — real-time data dikhao*
- *Todo complete karne pe chart automatically update ho*
- *Pomodoro timer ke baad productivity score update*
- *setInterval se auto-refresh (har 5 seconds mein)*

```
// Auto update har 5 seconds mein
setInterval(() => {
  chart.data.datasets[0].data.push(getNewData());
  chart.update();
}, 5000);
```

❓ ANALOGY

chart.update() = React ka setState() jaisa — UI refresh karta hai!

❓ CHAPTER 8: localStorage Integration

Page reload pe data delete na ho — isliye localStorage use karte hain!

```
// ☐ Data save karo
localStorage.setItem('tasks', JSON.stringify(tasks));

// ☐ Data load karo
let saved = localStorage.getItem('tasks');
let tasks = saved ? JSON.parse(saved) : [];

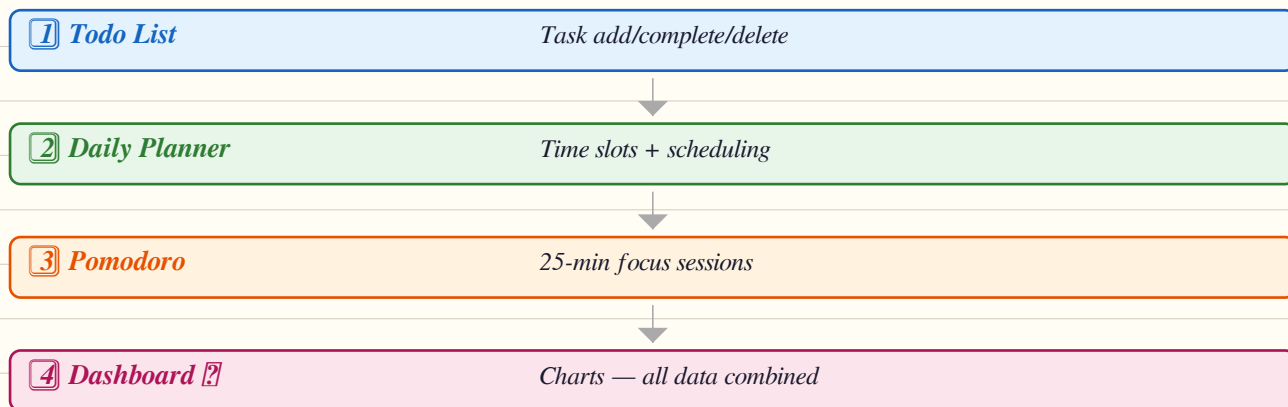
// ☐☐ Delete karo
localStorage.removeItem('tasks');
localStorage.clear(); // sab kuch delete
```

❓ Full Flow:

User Action → Update JS data → Save to localStorage

❓ CHAPTER 9: Real Dashboard Architecture

Tera final project ka blueprint ❓ — 4 modules connected together:



❓ Dashboard mein kya dikhega?

- Doughnut Chart → Completed vs Pending tasks %
- Line Chart → Weekly productivity trend
- Bar Chart → Time spent per day (Pomodoro sessions)
- Score Card → Today's productivity score (formula-based)

black CHAPTER 10: Advanced Concepts ●

📌 1. Streak System

Continuous productive days track karna — jaise GitHub contribution graph!

Formula: agar aaj kaam kiya aur kal bhi → streak++, warna reset

📌 2. Productivity Score

```
// 🟡 🟢 Score formula
function getScore(completedTasks, pomodoroSessions, streakDays) {
  let taskScore    = completedTasks * 10;
  let focusScore   = pomodoroSessions * 5;
  let streakBonus  = streakDays > 3 ? 20 : 0;
  return taskScore + focusScore + streakBonus;
}
```

📌 3. Weekly Analytics

- Best Day: sabse zyada productive din nikalo (Math.max)
- Worst Day: min score ka din — improvement needed
- Average: weekly average score calculate karo

```
let scores = [80, 45, 90, 60, 75, 30, 85];
let bestDay = Math.max(...scores); // 90
let worstDay = Math.min(...scores); // 30
let average = scores.reduce((a,b)=>a+b,0) / scores.length; // ~66
```

black CHAPTER 11: Canvas vs DOM

Feature	Canvas	DOM (HTML)
Speed	🚀 Fast (GPU-based)	🐢 Slow for heavy graphics
Rendering	Pixel-based drawing	Element-based layout
Animation	Super smooth 60fps	Can be choppy
Memory	Lightweight	Heavier (DOM nodes)
Interaction	Manual calculation	Built-in events
Chart.js	Canvas use karta hai ✔	—

[📌 KEY INSIGHT]

Chart.js Canvas use karta hai — isliye animations itni smooth hoti hain! 📌

purple CHAPTER 12: Project-Level Thinking 🧐

🧐 ✖ Beginner Says:

- "I made a todo app"
- "I added some charts"
- "It looks cool"

🧐 ✔ Advanced Developer Says:

- "Analytics dashboard with real-time tracking"
- "Productivity scoring with streak system"
- "Data-driven insights with Chart.js visualizations"

🧐 🔑 Key Differences to Remember:

UI vs System • Static vs Dynamic • Feature vs Insight

Jitna deep sochoge, utna better developer banoge! 🧐

A BONUS A: Colour Arrays — Har Bar Ka Alag Colour

Ek hi colour se sab bars same dikhte hain — boring! Array use karo:

```

    ● ● ●
datasets: [{
  label: 'Tasks per Day',
  data: [5, 12, 8, 15, 3],
  backgroundColor: [
    '#FF6384', // Mon - red
    '#36A2EB', // Tue - blue
    '#FFCE56', // Wed - yellow
    '#4BC0C0', // Thu - teal
    '#9966FF'  // Fri - purple
  ]
}]

```

```

    ● ● ●
// Dynamic colour generator — N items ke liye
function generateColors(n) {
  const palette = ['#FF6384', '#36A2EB', '#FFCE56', '#4BC0C0',
    '#9966FF', '#FF9F40', '#C9CBCF', '#7BC8A4'];
  return Array.from({length:n}, (_,i) => palette[i % palette.length]);
}
// Use: chart.data.datasets[0].backgroundColor = generateColors(tasks.length);

```

[WARNING]

Pie/Doughnut mein colour ARRAY zaruri hai — warna sab ek hi colour!

B BONUS B: Chart Destroy & Recreate

Ek canvas pe dobara Chart banane se ERROR aata hai — destroy pehle karo!

```

    ● ● ●
let myChart = null; // global variable rakhna ZARURI hai

function createChart(type) {
  if (myChart) { myChart.destroy(); } // Step 1: destroy
  let ctx = document.getElementById('myChart');
  myChart = new Chart(ctx, { // Step 2: recreate
    type: type,
    data: chartData,
    options: chartOptions
  });
}
// Button se chart type switch karo
document.getElementById('barBtn').onclick = () => createChart('bar');
document.getElementById('lineBtn').onclick = () => createChart('line');

```

● RULE: Chart variable ko GLOBAL rakho! ●

if (myChart) myChart.destroy(); phir naya banao

Bina destroy ke dobara banana = 'Canvas already in use' ERROR

C BONUS C: Tooltip Customization

Tooltip = hover pe jo popup dikhta hai. Fully customize kar sakte ho!

```
options: {
  plugins: {
    tooltip: {
      callbacks: {
        title: (items) => `Day: ${items[0].label}`,
        label: (item) => `Tasks: ${item.raw}`,
        afterLabel: (item) => item.raw > 10 ? 'Great day!' : 'Keep going'
      },
      backgroundColor: '#1A1A2E',
      titleColor: '#FFD54F',
      bodyColor: '#90CAF9',
      padding: 12,
      cornerRadius: 8
    }
  }
}
```

Callback	Kab call hota hai
<code>title(items)</code>	Tooltip ka heading — customize karo
<code>label(item)</code>	Har data point ki value line
<code>afterLabel(item)</code>	Value ke baad extra message dikhao
<code>footer(items)</code>	Tooltip ke neeche footer text

[KEY PROPS]

`item.raw` = actual number, `item.label` = x-axis naam — dono use karo!

D BONUS D: Chart Export — PNG Download

Chart ko PNG image ke roop mein download kar sakte ho — 3 lines ka code!

```
document.getElementById('downloadBtn').addEventListener('click', () => {
  let imageURL = myChart.toBase64Image(); // canvas se image URL
  let link = document.createElement('a');
  link.href = imageURL;
  link.download = 'my-chart.png';
  link.click(); // automatic download
});

// BONUS: Chart ko <img> tag mein bhi show kar sakte ho
document.getElementById('previewImg').src = myChart.toBase64Image();
```

[BONUS USE]

`toBase64Image()` Canvas ka data URL deta hai — kisi bhi img src mein use karo!

E BONUS E: Data Filter & Sort Before Charting

Raw data seedha chart mein mat daalo — pehle filter/sort karo!

```
let tasks = [
  { name: 'Task A', status: 'done',    score: 80 },
  { name: 'Task B', status: 'pending', score: 0  },
  { name: 'Task C', status: 'done',    score: 90 },
];

// 1. Sirf done wale tasks filter karo
let done = tasks.filter(t => t.status === 'done');

// 2. Score se sort karo (high -> low)
let sorted = done.sort((a, b) => b.score - a.score);

// 3. Chart ke liye extract karo
let labels = sorted.map(t => t.name);    // ['Task C', 'Task A']
let values = sorted.map(t => t.score);   // [90, 80]

// 4. Chart update karo
chart.data.labels = labels;
chart.data.datasets[0].data = values;
chart.update();
```

Operation	Code
Filter done	<code>tasks.filter(t => t.status === 'done')</code>
Sort high->low	<code>tasks.sort((a,b) => b.score - a.score)</code>
Get names	<code>tasks.map(t => t.name)</code>
Get values	<code>tasks.map(t => t.score)</code>
Count done	<code>tasks.filter(t=>t.done).length</code>
Group by day	<code>tasks.reduce((acc,t)=>{acc[t.day]=(acc[t.day] 0)+1;return acc;},{})</code>

F BONUS F: Todo <-> Chart Live Mapping (Full Code)

Todo complete karte hi chart automatically update ho — yahi real dashboard hai!

```
let tasks = JSON.parse(localStorage.getItem('tasks')) || [];
let chart;

window.onload = () => { renderTasks(); initChart(); };

function addTask() {
  let name = document.getElementById('taskInput').value.trim();
  if (!name) return;
  tasks.push({ id: Date.now(), name, done: false });
  save(); renderTasks(); updateChart();
}

function toggleTask(id) {
  tasks = tasks.map(t => t.id===id ? {...t, done:!t.done} : t);
  save(); renderTasks(); updateChart();
}

function save() { localStorage.setItem('tasks', JSON.stringify(tasks)); }

function getChartData() {
  let done = tasks.filter(t=>t.done).length;
  let pending = tasks.length - done;
  return {
    labels: ['Done', 'Pending'],
    datasets:[{data:[done, pending],
      backgroundColor:['#4CAF50','#FF7043']}]
  };
}

function initChart() {
  let ctx = document.getElementById('progressChart');
  chart = new Chart(ctx, { type:'doughnut', data: getChartData() });
}

function updateChart() { chart.data = getChartData(); chart.update(); }

function downloadChart() {
  let a = document.createElement('a');
  a.href = chart.toBase64Image(); a.download='progress.png'; a.click();
}
```

MASTER PATTERN:

Action -> Update State -> Save localStorage -> Update Chart

Todo CRUD + localStorage + Chart.js + Export = Full Dashboard

G BONUS G: Pomodoro -> Chart Integration

Har Pomodoro session complete hone pe chart mein time log karo!

```
let sessions = JSON.parse(localStorage.getItem('sessions')) ||
  { Mon:0, Tue:0, Wed:0, Thu:0, Fri:0, Sat:0, Sun:0 };

// Timer complete hone pe call karo
function onPomodoroComplete() {
  let days = ['Sun','Mon','Tue','Wed','Thu','Fri','Sat'];
  let today = days[new Date().getDay()];
  sessions[today] += 25; // 25 minutes add karo
  localStorage.setItem('sessions', JSON.stringify(sessions));
  updateTimeChart();
}

function updateTimeChart() {
  timeChart.data.labels = Object.keys(sessions);
  timeChart.data.datasets[0].data = Object.values(sessions);
  timeChart.update();
}
```

Flow: Pomodoro -> Dashboard

1. Timer starts (25 min countdown)
2. Timer ends -> onPomodoroComplete() call
3. sessions[today] += 25 (localStorage save)
4. updateTimeChart() -> timeChart.update()
5. Bar chart mein aaj ka bar grow karta hai!

H BONUS H: Error Handling in Chart Data

Galat data chart mein jaaye toh crash ya weird results — handle karo!

```
// Data sanitize karo — NaN, null, undefined hata do
function sanitizeData(arr) {
  return arr.map(val => { let n = Number(val); return isNaN(n) ? 0 : n; });
}

// localStorage se safely load karo
function loadData(key, fallback = []) {
  try {
    let raw = localStorage.getItem(key);
    return raw ? JSON.parse(raw) : fallback;
  } catch (e) {
    console.error('Load failed:', e);
    return fallback; // error aaye toh default value
  }
}

// Chart banane se pehle check karo
function safeCreateChart(data) {
  if (!data || data.length === 0) {
    console.warn('No data!'); return;
  }
  // ... chart code
}
```

Problem	Cause	Fix
Chart blank	Empty data array	Check <code>data.length > 0</code>
NaN in chart	String in <code>data[]</code>	<code>Number()</code> se convert karo
Crash on reload	<code>localStorage</code> null	<code>try/catch</code> + <code>fallback</code> value
Duplicate chart	Canvas reuse	<code>destroy()</code> pehle call karo
Wrong y-axis	Negative values	<code>min</code> : option use karo

I BONUS I: Chart.js v3 vs v4 — Key Differences

Different versions mein syntax thoda alag hai — confuse mat ho!

Feature	v3 Syntax	v4 Syntax
Axis config	<code>xAxes: [{}]</code> (array)	<code>x: {}</code> (object) ✓
Tooltip ns	<code>options.tooltips</code>	<code>options.plugins.tooltip</code>
Legend ns	<code>options.legend</code>	<code>options.plugins.legend</code>
Title ns	<code>options.title</code>	<code>options.plugins.title</code>
Tree shaking	No	Yes (smaller bundle)
Recommended	v3 still works	v4 use karo ✓

[VERSION CHECK]

Error aaye toh CDN URL check karo — v3 vs v4 syntax alag hai!

purple ? FINAL SUMMARY — *Tu Kya Seekh Gaya?*

? <i>Chart.js basics</i>	— Canvas, CDN, new Chart()
? <i>Data structure</i>	— labels, datasets, data array
? <i>Chart types & options</i>	— bar, line, pie, doughnut + styling
⚡ <i>Dynamic updates</i>	— push() + chart.update()
? <i>localStorage</i>	— Persistent data across reloads
? <i>Dashboard architecture</i>	— 4 modules integrated
? <i>Advanced concepts</i>	— Streak, score, weekly analytics
? <i>Canvas vs DOM</i>	— Why Chart.js is fast
? <i>Project-level thinking</i>	— System not just UI

? NEXT LEVEL — *Jab Tu Ready Ho*

- *Todo* → Chart data mapping (live connection)
- *Planner* → Time tracking logic
- *Full Real Dashboard build* (complete project)